NegGrad+ instability in BatchNorm networks

When the data to forget is concentrated in a single class

Khalil Bibi

October 2026

Contents

Goal **1**

1 Experimental setup **1**
1.1 Data and notation . . . 1
1.2 Model and training of O . . . 1
1.3 Weight initialisation . . . 2
1.4 BatchNorm . . . 2
1.5 Number formats . . . 3
1.6 NegGrad+ and the unlearning procedure . . . 3
1.7 Measures . . . 3
1.8 How we judge forgetting . . . 3

2 Step 1: original study and diagnostics **4**
2.1 The original study . . . 4
2.2 Diagnostic 1: BatchNorm memory or weights? . . . 4
2.3 Diagnostic 2: how the forget batch goes through BatchNorm . . . 5
2.4 Diagnostic 3: which part of BatchNorm? . . . 5

3 Step 2: replication with statistical tests **6**
3.1 Plan . . . 6
3.2 The statistic U . . . 7
3.3 Results . . . 7
3.4 CIFAR-10 / ResNet-18 specifics . . . 8

4 Step 3: which part of BatchNorm? **8**
4.1 Plan . . . 9
4.2 Measures . . . 9
4.3 Results . . . 9

5 Step 4: concentration or hard classes? **10**
5.1 Plan . . . 10
5.2 Measure: the early loss . . . 10
5.3 Model . . . 11
5.4 Results . . . 11

6 Step 5: is it the NegGrad+ objective? **12**
6.1 First attempt: the clipping burden . . . 12
6.2 Second attempt: the typical gradient size $\bar{G}$ . . . 13

7 Step 6: remedies for the instability **13**
7.1 How we measure the damage 13
7.2 Main remedy: per-condition tuning 14
7.3 The other two remedies 14

8 Limits and future work **15**

Goal

We try to show that **NegGrad+ becomes unstable in BatchNorm networks when the set to forget is concentrated in a single class**, whereas a **stratified** set of the same size causes no problem. We then look for which part of BatchNorm is responsible, whether the NegGrad+ objective plays a role, and whether the problem can be fixed.

1 Experimental setup

1.1 Data and notation

- **Fashion-MNIST**: 60,000 training images (28×28, greyscale, 10 classes). 5,000 are set aside for validation (500 per class), so $|D| = 55{,}000$. The 10,000 test images are used for evaluation only.
- D_f: the set to forget, always **600 images**. $D_r = D \setminus D_f$: the set to keep, so $D = D_f \cup D_r$.
- **Concentrated** (c0 to c9): the 600 images all come from one class, about 11% of it. The rest of the class stays in D_r: we want to forget *these* 600 boots, not the concept of a boot.
- **Stratified**: 60 images from each class.
- **Reference model** R: retrained from scratch on D_r only, with the same recipe as O. It is what we want to approach without retraining. (From step 2 onwards, R is mostly a yardstick for forgetting, not a measure of instability.)

1.2 Model and training of O

A small convolutional network: 3 blocks of 3×3 convolution (no bias) + BatchNorm + ReLU, with 32, 64 then 128 filters and two max-pooling layers, followed by two dense layers (256 then 10). About 1.7 M parameters and 3 BatchNorm layers.

- SGD, learning rate 0.05 with a cosine schedule, momentum 0.9, weight decay 5×10^{-4};
- 15 epochs, batch size 128, no data augmentation;
- about **93%** test accuracy. One model O per seed.

1.3 Weight initialisation

All experiments use **PyTorch's default initialisation**: no custom initialisation was written (no `nn.init` or `kaiming` call in the notebooks).

- **Convolutions and dense layers**: each weight is drawn uniformly in $[-b, b]$ with $b = 1/\sqrt{n}$, where n is the number of inputs of the neuron. This gives $\mathrm{Var}(w) = b^2/3 = 1/(3n)$.
- **Biases**: convolutions have none (the β of the following BatchNorm plays that role); dense-layer biases are also drawn uniformly in $[-1/\sqrt{n}, 1/\sqrt{n}]$.
- **BatchNorm**: the learned parameters start at $\gamma = 1$ and $\beta = 0$; the buffers start with a stored mean of 0 and a stored variance of 1.

Why the variance matters. If the signal variance grows from layer to layer, the output logits become huge: one class gets 100% after the softmax, and losses and gradients become huge. If it shrinks towards 0, all logits are close to 0: about 10% per class, and tiny gradients.

Kaiming (He). Kaiming uses $\mathrm{Var}(w) = 2/n$ (the 2 compensates for the half of the signal that ReLU zeroes), which keeps the variance constant across layers. PyTorch's default gives $1/(3n)$, six times less: without normalisation the variance would shrink by about $\times 6$ per layer.

Why it is not a problem here. Every convolution is followed by a BatchNorm that resets the scale to 1, so the shrinkage does not accumulate; only the two final dense layers feel it. The same holds for the ResNet-18 of step 2: no special initialisation, including around the skip connections.

Seeds. The seed is fixed right before the model is built, so the same seed gives the same starting weights bit for bit. Unlearning methods start from a **copy of** O; each R has its own initialisation seed.

1.4 BatchNorm

BatchNorm works **per channel**. For a channel c, it computes the mean μ_B and the variance σ_B^2 over **all images of the batch and all positions** ($N \times H \times W$ values), then:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma\,\hat{x} + \beta$$

where γ and β are learned by gradient descent (they let the network re-stretch and re-shift the values, or even undo the normalisation if that helps).

- **Training mode**: the layer normalises with the statistics of the current batch, and the gradient flows through μ_B and σ_B^2. It also updates its buffers at every batch:

  $$\mu_{\text{stored}} \leftarrow 0.9\,\mu_{\text{stored}} + 0.1\,\mu_B \qquad \text{(same for the variance)}$$

  The buffers are *not* learned: they are measured.
- **Evaluation mode**: the layer normalises with the buffers, which no longer change, then applies γ and β. It becomes a fixed affine map per channel.
- **Recalibration**: the buffers are recomputed exactly on the whole of D_r (true mean and variance), layer by layer, without touching the weights, γ or β. The images to forget are never used.

1.5 Number formats

All training runs in **FP32**; there is no half-precision training. On an A100, PyTorch runs convolutions in **TF32** by default (internal rounding to about 3 significant digits), hence check tolerances of 10^{-5} per layer and 10^{-4} relative on the full network. Sensitive checks (buffers, gradient norms) are computed in **FP64**. FP16 was only used to *store* some descriptive measures in the study of step 3 (an overflow above 65,504 was fixed by a dated amendment).

1.6 NegGrad+ and the unlearning procedure

We try to forget D_f with the loss

$$L = \alpha\,\mathrm{CE}_r - (1 - \alpha)\,\mathrm{CE}_f$$

The loss goes *down* on D_r and *up* on D_f. Since $\mathrm{CE} = -\log p$ has no ceiling (as $p \to 0$, $-\log p \to \infty$), nothing stops the push.

- One epoch over D_r = **425 steps**. At each step: a batch of **128** images from D_r and a batch of **64** images from D_f (successive permutations of D_f, so each image is seen about 45 times).
- SGD, constant learning rate 0.01, momentum 0.9, weight decay 5×10^{-4}.
- Forget weight $1 - \alpha \in \{0.001, 0.01, 0.05\}$, 0.01 as the main value.
- The global gradient norm is **clipped** at 5: a safety belt against huge steps when CE_f runs away.

1.7 Measures

Measure	Definition	Note
Divergence	$\mathrm{CE}_f > 10$ at some step, or a numerical failure (NaN, inf)	$\mathrm{CE}_f > 10$ means the (geometric) mean probability of the true class over the batch is $< e^{-10} \approx 0.005\%$. Chance gives CE = $\ln 10 \approx 2.3$.
Collapse	Test accuracy $\leq 20\%$, or $\geq 80\%$ of the predictions in a single class	Measured with the native buffers (the model as it is).
S_{mean}	Mean over steps of $\log_{10} \max(\mathrm{CE}_f, 10^{-7})$	The larger S_{mean}, the more unstable the run.

1.8 How we judge forgetting

Forgetting can never be checked directly: a model has forgotten D_f if it cannot be told apart from R, the model that never saw D_f. And forgetting does not mean being wrong: on concentrated sets, R still recognises ~93% of the images to forget, because it learned the class from the other images. A model that drops far below that does too much, which is also detectable. This report uses three yardsticks, always with recalibrated buffers:

- **accuracy on D_f compared with R**: above R the model does not do enough; far below, it breaks;
- **forgetting strength** E = mean over D_f of $\log_{10} p_O(y \mid x) - \log_{10} p_{\text{model}}(y \mid x)$: how much the confidence in the true class dropped relative to O, in decades; E_R is the same quantity for R;
- **a membership-inference attack** (once, exploratory, on Tiny ImageNet): an attacker tries to guess whether an image was in the training set; an AUC of 0.5 means they cannot.

These yardsticks check that a method that does not break still does something. The report never claims that a model has *erased* D_f.

2 Step 1: original study and diagnostics

Caveat

These analyses are **post hoc**: the classes and variants were chosen after seeing the original study. They generate hypotheses, which are then tested on new models (steps 2 to 4).

2.1 The original study

Over 5 seeds we ran 11 forget sets: one per class (10 concentrated sets) and one stratified set, 600 images each. That is **55 NegGrad+ runs**.

Forget set	Divergences (/5)	Collapses (/5)
c1, c2, c4, c6	5	5
c5	4	3
c7	5	3
c0, c3, c8, c9	0	0
Stratified	0	0
Total	**29 / 55**	**26 / 55**

Reading

26 of 55 runs collapse (final accuracy $\leq 20\%$) and 29 diverge. But this **only concerns concentrated sets** (never the stratified one), and always the same six classes: 1, 2, 4, 6, and sometimes 5 and 7. This is not forgetting, it is an instability. So we look at what happens inside BatchNorm.

2.2 Diagnostic 1: BatchNorm memory or weights?

We replay the 55 runs **bit for bit** (same O, same batches, same order), changing only how the buffers are handled. In the original version **V0**, at each step:

```
1.  out_r = net(keep_batch)   // BN: statistics of the keep batch, updates its
buffers
2.  out_f = net(forget_batch) // BN: statistics of the forget batch, updates its
buffers
3.  loss = α·CE(out_r) − (1−α)·CE(out_f)
4.  one backward pass on this loss, clipping at 5, then weight update
```

- **V1c**: on line 2, the forget batch no longer updates the buffers.
- **V2**: after V0 unlearning, the buffers are recalibrated on the whole of D_r.
- $\alpha = 1$: same procedure without the forget term, to see whether training alone can cause the collapse.

Version	Test accuracy (on the 26 collapsed runs)
V0	10% (median)
V1c	69% (median)
V2 (recalibrated)	74% (median)
$\alpha = 1$	never collapses (minimum 91%)

Reading

The forget term is necessary ($\alpha = 1$ breaks nothing). Buffers polluted by the forget batches explain a large part of the measured collapse, but not all of it: even recalibrated, the models stay around 74% instead of 93%, so **the weights are damaged too**. On c1, CE_f reaches 1.9 to 3.0×10^5, with 68 to 74% of steps clipped.

2.3 Diagnostic 2: how the forget batch goes through BatchNorm

Same 5 seeds, the 6 collapsing classes, hence **30 runs per variant** ($1 - \alpha = 0.01$); we count divergences.

- **Variant A**: the same as V0.
- **Variant B**: the keep and forget batches go through the network together:

```
1.  out = net(keep_batch + forget_batch)  // BN: statistics of the combined batch
(128 + 64),
                                          // updates its buffers
2.  out_r = out[0:128], out_f = out[128:]
3.  loss = α·CE(out_r) − (1−α)·CE(out_f)
4.  one backward pass on this loss, clipping at 5, then weight update
```

- **Variant C**: BatchNorm stays in evaluation mode (with O's initial buffers), while γ and β are still updated by the backward pass.

Variant	Divergences	Test accuracy	Accuracy on D_f (c6)	
A	29 / 30	collapsed	–	
B	0 / 30	within ∼1.5 points of O	94.5%	(R: 79.5%)
C	0 / 30	within ∼1.5 points of O	93.0%	(R: 79.5%)

Reading

B and C go from 29 divergences to 0. This indicates that **normalising the forget batch with its own one-class statistics** (and letting the gradient flow through them) plays a role in the divergences during training. Note that divergence is measured in training mode, where the buffers are not used for normalisation, so buffer pollution is not what matters here. And it is a hint, not a proof: B and C change several things at once.
Forgetting, however, is very weak: on the images to forget, B and C stay close to O, not to R. They behave almost like fine-tuning: the model still recognises these images very well.

2.4 Diagnostic 3: which part of BatchNorm?

We now probe what drives the instability by changing BatchNorm in several ways: whether μ and σ^2 are constants in the backward pass, what the forget batch is normalised with, and whether the forget batch updates the buffers. We take **c1** (which diverges) and **c8** as a **stable control** (0 divergences under A), with 5 seeds, so 10 runs per variant. The keep-batch pass is unchanged; only the forget-batch pass changes:

Variant	Forget batch normalised with	Gradient through stats	Updates buffers
A	its own statistics	yes	yes
E2	its own statistics	yes	no
E1D	its own statistics	no (detached)	no
E1	the current buffers (updated by the keep batch only)	no	no
E1-O	O's frozen buffers	no	no

Variant	Div. c1	Max CE_f (c1)	Test accuracy c1	Div. c8
A	5/5	1.9 to 3.0×10^5	10%	0/5
E2	5/5	same as A	69 to 79%	0/5
E1D	5/5	11 to 34	32 to 88% (85 to 89% recalibrated)	0/5
E1	0/5	< 0.1	92.4 to 93.3%	0/5
E1-O	5/5	up to 1.9×10^{16}	10%	5/5

On c1, E1 still recognises 99.3 to 99.8% of the images to forget, but R also recognises **98.7%** of them (the trousers left in D_r are enough). So we cannot say that E1 fails to forget on this class.

Reading

We conclude that:

1. letting the gradient flow through the forget batch's statistics **hugely amplifies** the blow-up ($\sim 10^5$ for A versus 11 to 34 for E1D);
2. normalising with the forget batch's own statistics **coincides** with crossing the threshold (E1D crosses it, E1 does not);
3. frozen statistics from O also explode, even on c8: there is no simple "fixed versus batch statistics" story.

E2 diverges exactly like A (same weights) but collapses much less at evaluation: buffer updates by the forget batch affect *evaluation*, not divergence.

Note on gradients. We also looked post hoc at gradient norms during these runs. The keep-batch and forget-batch gradients do not conflict (cosine ≈ 0). And the gradient norm blows up about 28 steps (median) *after* CE_f has crossed 10: it is a symptom of divergence, not an early warning.

3 Step 2: replication with statistical tests

We now test these diagnostics statistically, with **new models** (seeds derived differently) and a protocol written **before** running: 3 studies, 1,141 units, 6 h on an A100, 9,801 checks passed.

3.1 Plan

- **Study 1**: Fashion-MNIST, BatchNorm CNN, variants A and C, 5 seeds.
- **Study 2**: the same CNN with **GroupNorm** instead of BatchNorm. GroupNorm normalises each image on its own, without looking at the batch. 5 seeds.
- **Study 3**: CIFAR-10 (50,000 colour images, 32×32) with a BatchNorm ResNet-18, variants A and C, 3 seeds.

For study 1 we reuse the recipe of O (15 epochs, 55k images, one O per seed). Each seed has 11 ways to forget (10 classes + stratified), and for each one:

- a reference model R (O retrained from scratch without the forget set);
- a fine-tuning run as a baseline;
- NegGrad+ with variants A and C, each at 3 forget weights (0.001, 0.01, 0.05).

That is $11 \times 2 \times 3 = 66$ NegGrad+ runs per seed, 330 over 5 seeds.

3.2 The statistic U

For each run we compute S_{mean} (mean $\log_{10} \text{CE}_f$): the larger, the more unstable. We compare each concentrated run with the **paired** stratified run (same seed, variant, weight and O):

$$\psi_c = \begin{cases} 1 & \text{if the concentrated run is more unstable} \\ 0.5 & \text{if tied} \\ 0 & \text{otherwise} \end{cases}$$

A run with a numerical failure always counts as the more unstable (none occurred). Then:

$$U_s = \text{mean of } \psi_c \text{ over 10 classes} \times 3 \text{ weights} = 30 \text{ comparisons, for seed } s$$

- U says how often concentrated runs are more unstable than the stratified reference. $U = 0.5$: no effect; $U = 1$: always more unstable.
- **Why per seed**: the 11 runs of a seed share the same O, so they are not independent; counting them separately would fake precision. We get one U per seed and run a **t test** over seeds.

Main hypotheses: $U_A - 0.5 > 0$ (under A, concentrated runs are more unstable) and $U_A - U_C > 0$ (the effect is stronger under A than under C); for study 2, $U_{GN} - 0.5 > 0$.

p-value. The probability of a result *at least as extreme* if there were no effect. It is not the probability that the result is due to chance.

$p_{\textbf{Holm}}$**.** With several tests at once, the risk that at least one "succeeds" by luck grows (with 10 tests at 5%, about 40%). Holm's correction adjusts the p-values to keep that overall risk at 5% (one family $\{H_1, H_{2a}\}$ per study).

TOST. "No difference detected" does not mean "no effect". The TOST shows that the mean loss gap concentrated − stratified (Δ_{mean}, in decades) is **smaller than a margin** of 1 decade: the 90% confidence interval must lie entirely within $[-1, 1]$.

3.3 Results

Study	**Test**	**Estimate [95% CI]**	$p_{\textbf{Holm}}$	**Conclusion**
1 (BN)	$U_A - 0.5$	0.36 [0.33, 0.39] ($U_A = 0.86$)	1.7×10^{-5}	effect
1 (BN)	$U_A - U_C$	0.37 [0.30, 0.45]	1.5×10^{-4}	effect
2 (GN)	$U_{GN} - 0.5$	−0.033 [−0.075, 0.008]	0.089	none detected
3 (CIFAR)	$U_A - 0.5$	0.5 ($U_A = 1$ in all 3 seeds)	–	not computable
3 (CIFAR)	$U_A - U_C$	0.43 [0.35, 0.52]	0.0039	effect

In study 3, $U_A = 1$ in all 3 seeds: the variance is zero, so the t test cannot be computed, but every concentrated run is more unstable.

TOST (margin ±1 decade)	$\Delta_{\textbf{mean}}$ **[90% CI]**	**Equivalent to 0?**
Study 1, A	2.39 [2.32, 2.47]	no
Study 1, C	−0.14 [−0.24, −0.04]	yes
Study 2, GroupNorm	−0.14 [−0.20, −0.08]	yes
Study 3, A	2.12 [2.08, 2.15]	no
Study 3, C	0.07 [0.04, 0.10]	yes

Reading

Under A, concentrated runs are almost always more unstable (on average 2.39 decades more loss, about 250 times more). The effect vanishes under C **and with GroupNorm**, and the TOST shows it is negligible there: what matters is normalising **with batch statistics**. On Fashion-MNIST at 0.01, A diverges in 30 of 50 concentrated runs (24 collapses), C in none; and they are exactly the same six classes as in step 1, with brand-new models.

3.4 CIFAR-10 / ResNet-18 specifics

Training for 30 epochs (learning rate 0.1, cosine) with augmentation (random crop with padding 4 and horizontal flip); 348 steps per epoch and 490 images to forget (49 per class when stratified). The conclusions of study 1 carry over to a larger architecture, with some differences:

- under A, concentrated sets diverge 0/30, 30/30 and 30/30 times at weights 0.001, 0.01 and 0.05, but there is **no collapse**: divergence and collapse are different things;
- the accuracy loss is much smaller than with the small CNN, and recalibration recovers almost all of it except at 0.05:

Test-accuracy loss of A vs O	**0.001**	**0.01**	**0.05**
Native buffers	5.6 pts	7.2 pts	28.5 pts (worst run: 43.9%)
After recalibration	1.5 pts	1.8 pts	7.7 pts

One could speculate that skip connections limit how far the BatchNorm instability propagates, but this is an *untested hypothesis*.

Caveat

The analysis notebook was not committed before it was run; 5 and 3 seeds, so the results are exploratory according to the protocol itself; and class identity and concentration are still confounded (always the same classes).

4 Step 3: which part of BatchNorm?

We now study the mechanism of the BatchNorm instability, in particular which part produces it. Two questions:

(Q1) Does letting the gradient flow through BatchNorm's statistics amplify the instability (keeping μ and σ^2 detached from the backward pass or not)? $\rightarrow$ compare A and E1D.

(Q2) Does normalising with the forget batch's statistics rather than the current buffers amplify it? $\rightarrow$ compare E1D and E1.

Variant	**Forget batch normalised with**	**Gradient through stats**	**Updates buffers**
A	its own statistics	yes	yes
E2	its own statistics	yes	no
E1D	its own statistics	no	no
E1	the current buffers	no	no

4.1 Plan

A small novelty: several draws per class. For each seed:

- 3 draws of 600 images for each class (30 concentrated sets);
- 10 draws for the stratified set;
- hence 40 forget sets per seed.

With 3 weights and 5 seeds that gives **600 units** (450 concentrated, 150 stratified). Seeds are derived differently from step 2, so the O models are new. A and E2 have **exactly the same weights** (only their buffers differ), so they form a single trajectory: 3 trajectories per unit (A/E2, E1D, E1), plus FT, R, O and controls, **2,445 runs** in total (5.39 h).

4.2 Measures

- **A versus E1D**: the same S_{mean} over the 425 steps. This is comparable because, at equal weights, A and E1D have the same forward pass. Hypothesis H_B: in concentrated sets, is A more unstable than E1D?
- **E1D versus E1**: their forward passes differ (the buffers are not updated by the same things), so we use a common measure. Every 25 steps, we stop and take a clean buffer computed in the same way for both (updated by keep batches only); with it, we measure the loss on the 600 images to forget. 17 stops ($25 \times 17 = 425$ steps), and we average $\log_{10} \text{CE}_f$ over them. This is H_F.
- θ = share of concentrated units where the first variant is more unstable; $\iota = \theta_{\text{conc}} - \theta_{\text{strat}}$ says whether the effect is stronger in concentrated than in stratified sets (I_B and I_F). Holm over the 4 tests, continuous gaps in decades, equivalence margin ± 1 decade.

4.3 Results

Test	θ **or** ι	$p_{\textbf{Holm}}$	**Continuous gap [95% CI]**	**Conclusion**
H_B: A vs E1D	0.82	3.7×10^{-5}	+0.83 [0.81, 0.86]	supported
H_F: E1D vs E1	0.65	7.5×10^{-4}	+0.04 [−0.08, 0.16]	weakened (gap equivalent to 0)
I_B: A/E1D effect stronger when concentrated?	+0.33	4.0×10^{-4}	+0.83 [0.81, 0.86]	supported
I_F: E1D/E1 effect stronger when concentrated?	+0.40	7.5×10^{-4}	+0.89 [0.70, 1.09]	supported, but see below

Divergences (out of 150 concentrated, 50 stratified)	**0.001**	**0.01**	**0.05**
A, concentrated	12	90	150
E1D, concentrated	12	89	121
E1, concentrated	0	60	150
E1, stratified	0	0	47
A and E1D, stratified	0	0	0

Reading

1. **Q1: yes.** On concentrated sets, A reaches a higher forget loss than E1D in 82% of units, with mean losses about **7 times larger** (+0.83 decade), and the gap grows with the forget weight (+0.01, +0.75 then +1.73 decade). It is specific to concentration: on stratified sets the continuous gap is close to zero (I_B).
2. **Q2: weak effect.** E1D is more unstable than E1 in 65% of cases, but the continuous gap is equivalent to 0. The positive I_F comes mostly from **stratified sets at 0.05**, where E1 diverges 47 times out of 50; on concentrated sets, the sign changes with the weight (+0.66 decade at 0.01, −0.61 at 0.05). So we cannot say the E1D-versus-E1 effect is stronger on concentrated sets.
3. **No variant is a cure**: on concentrated sets at 0.05, A diverges 150/150, E1D 121/150 and E1 150/150.

5 Step 4: concentration or hard classes?

We now want to know whether the problem comes from some classes being harder to forget than others, or from concentration itself. Until now (except on ResNet-18), the same six classes always collapse or diverge: maybe "concentrated" just meant "contains a hard class".

5.1 Plan

We take 10 new seeds (401 to 410): each gives a new O, a new image order and a new initialisation. We stay with variant A at $1-\alpha = 0.01$, with C as a secondary variant. For each seed we build 68 sets of 600 images:

- 10 pure sets: 600 images from a single class;
- 45 pairs: 300 + 300 from two different classes;
- 12 five-class sets: 120 per class;
- 1 stratified set.

So $68 \times 10 = 680$ runs. Concentration is measured by the label entropy $H = \log_2(\text{number of classes})$: 0 bits for one class, 1 for two, 2.32 for five, 3.32 for ten.

5.2 Measure: the early loss

$$S = \text{mean of } \log_{10} \mathrm{CE}_f \text{ over the first 24 steps (0 to 23)}$$

This choice was fixed **in advance** on earlier traces: no divergence happened before step 24 (the threshold of 10 applies to CE_f), and S already separated the runs that would diverge from the others (AUC 0.92). So we measure the cause *before* the blow-up, not the size of the blow-up.

5.3 Model

We predict what each mix would give if only the classes mattered. Each class c has an effect x_c, and for a mix with proportions p_c the prediction is a power mean:

$$M_\lambda = \Big(\sum_c p_c \, x_c^\lambda \Big)^{1/\lambda} \qquad \lambda = 1 : \text{arithmetic}; \quad \lambda \to 0 : \text{geometric}; \quad \lambda = -1 : \text{harmonic}$$

Rather than guessing λ, we let it vary between -3 and 3. The model is:

$$S = \log_{10} M_\lambda(p\,; x) + \gamma \cdot H + \text{noise}$$

If class identity explains everything, $\gamma = 0$; if spreading a request over more classes helps beyond the classes it contains, $\gamma < 0$.

```
for each seed s = 1, ..., 10:
    fit (x0...x9, λ, γ) on its 68 sets by non-linear least squares
    keep γ_s
t statistic over the 10 γ_s
p-value by parametric bootstrap:  2,000 simulated experiments with γ = 0,
    each refitted; p = share of simulated t at least as extreme as the real one
```

The bootstrap does **not** estimate x, λ and γ (least squares does that): it calibrates the threshold of the test on γ, because λ is estimated and the Student table is then no longer reliable. The null hypothesis is "concentration has no effect on the early loss".

5.4 Results

Result	**Value**
γ	-0.246 decade per bit $[-0.263, -0.228]$
p (bootstrap)	0.001, the smallest this bootstrap can give
Seeds with $\gamma < 0$	10 out of 10
Estimated λ	between 0.34 and 1.09
γ under C (secondary)	-0.004, about 0

Classes in D_f	**Div. (A)**	**Collapse (A)**
1	62/100	54/100
2	67/450	22/450
5	0/120	0/120
10	0/10	0/10

Reading

There is a real effect: the smaller H (the more concentrated the set to forget), the larger the early loss, **beyond what its classes predict**. From 1 to 10 classes (3.32 bits), the early loss drops by $0.246 \times 3.32 \approx 0.82$ decade, about **6.6 times** less than the classes predict. Under C (BatchNorm in evaluation mode), the effect disappears.

Caveat

A single forget weight, a single model and a single dataset; the model cannot represent interactions specific to some pairs of classes; and the measure is the early loss, not divergence or forgetting directly.

6 Step 5: is it the NegGrad+ objective?

We now want to know whether the instability comes only from BatchNorm, or whether the NegGrad+ objective itself is part of the problem. We compare NegGrad+ with an unlearning method whose objective is bounded: **random labels** (RL). Each image to forget gets a wrong label drawn among the other 9 classes (fixed once per seed and set), and the model is trained towards it:

$$L = \alpha \, \mathrm{CE}_r + (1 - \alpha) \, \mathrm{CE}(x_f, \text{ wrong label})$$

Unlike NegGrad+, this loss has a **floor**. Take a boot to forget, which RL labels "sneaker":

NegGrad+: push $-\log p(\text{boot})$ **up**		**RL: push** $-\log p(\text{sneaker})$ **down**	
$p(\text{boot})$	loss	$p(\text{sneaker})$	loss
0.99	0.01	0.01	4.6
0.1	2.3	0.5	0.69
10^{-4}	9.2	0.99	0.01
$\to 0$	$\to +\infty$	$\to 1$	$\to 0$

With NegGrad+ there is no finish line: $p(\text{boot})$ can always get closer to 0, so the loss keeps rising and the gradient keeps pushing (only clipping slows it down). With RL the loss cannot go below 0: once the model already says "sneaker", that image's term and its gradient become tiny, so **the push dies out on its own**. RL is "walk to the wall and stop"; NegGrad+ is "get as far from the wall as you can".

Note: this does not mean $p(\text{sneaker})$ rises at every step. Each step only sees a batch of 64 images, the step has a finite size (momentum, clipping), and above all the keep term (99% of the loss) pulls the other way, since D_r contains other boots labelled "boot". The floor only says the loss is bounded below and cannot run away. (We did not track $p(\text{sneaker})$ step by step: we measured gradients, divergences and accuracy.)

Everything else is identical: same O, same forget sets, same batches in the same order, same BatchNorm treatment (variant A), same learning rate and same clipping at 5. The objective is the only thing that changes: if NegGrad+ blows up and RL does not, it can only come from there.

6.1 First attempt: the clipping burden

We first measured the **clipping burden** B = the share of the 425 steps where the gradient norm exceeds 5 and must be clipped. $B = 0$: never too large; $B = 0.5$: one step in two. For each method, the effect of concentration is $\Delta = B_{\text{concentrated}} - B_{\text{stratified}}$, and we compare the two methods with

$$\theta_B = \Delta_{\text{NegGrad+}} - \Delta_{\text{RL}}$$

(10 seeds, 5 of them reused from step 2, 782 runs). We find $\theta_B = 0.30$ [0.29, 0.32]: on concentrated sets, NegGrad+ has to clip about 30% more steps than RL.

Caveat

Under RL, B is stuck at 0 (4 clipped steps in 330 runs), on concentrated and stratified sets alike, so $\Delta_{\text{RL}} \approx 0$. But this does not prove that RL is unaffected: if concentration takes the gradient norm from 1 to 2, it stays below the threshold of 5 and B sees nothing. This is a **floor effect**, like a fever alarm that only rings above 40 °C. A continuous measure, without a threshold, was needed.

6.2 Second attempt: the typical gradient size $\bar{G}$

We take 10 new seeds (101 to 110), so new models and nothing reused. At each step we measure the norm of the gradient of the full loss before clipping, and:

$$\bar{G} = \text{mean over the 425 steps of } \log_{10}(\text{gradient norm})$$

$\bar{G}$ measures how hard the optimiser is pushed; it is not an instability measure in itself. $+1$ in $\bar{G}$ means gradients 10 times larger, $+0.1$ about 26% larger. For each seed and each method:

$$\text{shift} = \bar{\bar{G}}_{\text{10 pure sets}} - \bar{G}_{\text{stratified}}, \qquad \theta = \text{shift}_{\text{NegGrad+}} - \text{shift}_{\text{RL}}$$

This gives one θ per seed, 10 values, and a t test at weight 0.01 (main test); weights 0.001 and 0.05 are secondary, with Holm. Before running, we simulated the analysis on data with no effect to check that it does not cry wolf: 4.4% to 5.9% false positives, so fine.

Result (weight 0.01)	**Value**
Shift under NegGrad+	0.705 decade (gradients ∼5 times larger on concentrated sets)
Shift under RL	0.053 decade (gradients ∼13% larger)
θ	0.652 decade [0.607, 0.696], $t = 33.3$
Seeds with $\theta > 0$	10 out of 10

Forget weight $1-\alpha$	**0.001**	**0.01**	**0.05**
θ	−0.015	0.652	2.644
Test-accuracy loss on concentrated sets, NegGrad+	1.3 pts	46.8 pts	82.9 pts
Test-accuracy loss on concentrated sets, RL	1.0 pt	2.0 pts	2.9 pts

Reading

Concentration disturbs both methods, but NegGrad+ far more: the unbounded objective turns the small disturbance caused by BatchNorm into a blow-up. At 0.001 the push is too weak to matter. This does not show that RL is unstable (its small shift never comes with a divergence), nor which part of the network is responsible.

7 Step 6: remedies for the instability

We now know that the problem comes from BatchNorm on one-class batches and from the unbounded NegGrad+ objective. So we try three remedies, each aimed at a different suspect.

7.1 How we measure the damage

We first need a measure of "how damaged the model is":

$$U = \text{accuracy of } O \text{ on } D_r - \text{accuracy of the unlearned model on } D_r$$
$$\text{penalty} = \text{mean of } U \text{ over the 10 concentrated sets} - U \text{ of the stratified set} \quad \text{(per seed)}$$

Both models are recalibrated on D_r before measuring U, so that U measures damage in the weights, not buffer pollution. The effect of a remedy is I = penalty of plain NegGrad+ − penalty with the remedy.

Caveat

A small penalty may just mean that almost nothing was done: a model that forgets nothing damages nothing. So we always look at the **forgetting strength** E alongside it, and at the target R, which still recognises ∼93% of the images to forget on concentrated sets. The goal is not 0%.

7.2 Main remedy: per-condition tuning

Maybe NegGrad+ is not fragile at all, and the default setting (learning rate 0.01 and $1-\alpha = 0.01$) is simply bad for concentrated sets. So we look for the best setting for each class, among 20 configurations:

$$\text{lr} \in \{0.001, 0.003, 0.01, 0.03\} \times 1-\alpha \in \{0.001, 0.003, 0.01, 0.03, 0.05\}$$

Tuning uses 3 seeds (447 to 449) and a rule fixed in advance:

1. **stable** = no divergence and no collapse on the 3 seeds;

2. **feasible** = stable *and* strength $E \geq$ that of R;
3. among feasible settings, pick the best (recalibrated) validation accuracy;
4. if no setting is feasible: among stable ones, pick the one that forgets the most.

Evaluation then uses 12 other seeds (450 to 461) that were not used for tuning; otherwise we would only measure the luck of having found a setting that works on those 3 seeds. The 12 seeds come from a power simulation. We compare tuned and default NegGrad+ with a t test on the penalty and a TOST with a margin of ±2 points.

	Default NegGrad+	**Tuned NegGrad+**
Concentration penalty	14.2 pts [13.0, 15.5]	1.07 pts [0.88, 1.27]
TOST (margin ±2 pts)	–	equivalent to 0
Divergences on concentrated sets	71 / 120	2 / 120
Collapses on concentrated sets	65 / 120	–

On the TOST: the interval [0.88, 1.27] excludes 0, so a small penalty remains; but it lies entirely below 2, so that penalty is small. Both are true at once.

Reading

Tuning removes almost all of the penalty. However, only 3 classes out of 10 (c4, c8, c9) have a setting that is both stable and as strong as R. The stratified set is stable under all 20 settings, concentrated sets under 1 to 16 depending on the class. For the other 7 classes, the rule fell back to the strongest stable setting, which forgets less than R. On the evaluation seeds, R's strength is reached in only 20% of concentrated runs (67% of stratified ones).
In short: **to stay stable on concentrated sets, NegGrad+ must push less than it would need to forget as much as** R.

7.3 The other two remedies

- **Frozen target**: instead of pushing the loss to infinity, the model aims at a precise answer. For each image to forget, the target is O's output with the true class removed (masked, then renormalised). It is a bounded objective like RL, but more natural (a boot looks like a sneaker, not like a bag). On 12 seeds (421 to 432), the penalty drops by 16.2 points [14.1, 18.4] relative to NegGrad+, with 0 collapses against 70/120. But the model still recognises ∼97% of the images to forget, and it reaches its target only in training mode, inside one-class batches: once recalibrated, the change is gone.
- **RD64**: NegGrad+ is kept, but the forget batch is normalised with the statistics of 64 images of the (mixed) keep batch, detached from the gradient and without updating the buffers. On 14 seeds (433 to 446), the penalty drops by 13.7 points [12.6, 14.7] in all 14 seeds, and divergences go from 84/140 to 0/140. But since three things change at once, we cannot tell which one does the work.

Accuracy on the images to forget (concentrated, recalibrated)	
Plain NegGrad+ (A): goes too far, it breaks	80.2%
R: the target	93.1%
NegGrad+ with RD64	96.7%
Frozen target	∼97%
Fine-tuning (does almost nothing)	98.5%

Reading

The three remedies avoid the breakdown, but mostly because they forget less: they end up next to fine-tuning, not next to R. They are remedies **for the instability**, not remedies **for forgetting**: they remove divergences and the penalty, but none of them forgets as much as R. Making NegGrad+ both stable and as effective as retraining on concentrated sets remains an open problem.

8 Limits and future work

- We never measure true erasure of the data: there is a single, exploratory membership-inference attack, on Tiny ImageNet.
- BatchNorm is the main suspect, but no mechanism was isolated: most variants change several things at once.
- This is mostly a small CNN on Fashion-MNIST, with 3 to 14 seeds. On Tiny ImageNet (200 classes, ResNet-18), the effect goes in the same direction but is much weaker ($\theta = 0.029$ decade).
- The studies are not independent replications (same data split).
- Future work: run the PCGrad study already written (never executed), replace BatchNorm with GroupNorm or LayerNorm across the whole chain, actually measure the forgetting of the remedies, and try to predict risky deletion requests in advance.